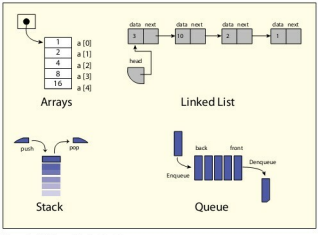


Figure 1: Different data structures

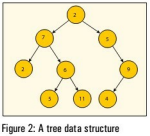


Figure 2: A tree data structure

Arrays: This is a data structure that can store a fixed-size sequential collection of elements of the same type. Elements are accessed using indexes.

Linked List: This data structure consists of a group of nodes, which together represent a sequence. Under the simplest form, each node is composed of data and a pointer to the next node. This structure allows for efficient insertion or removal of elements from any position in the sequence.

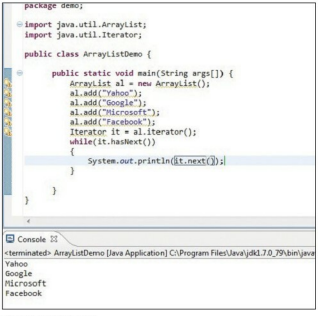
Stack: This is a data structure that implements the LIFO (Last In First Out) paradigm. It is like a pile of plates -- the one put in last is removed first. It can be implemented both through arrays and Linked List. Push and Pop are the two main operations on the stack. This is basically used in recursion.

Queue: This is a FIFO (First In First Out) data structure. It is similar to a queue of people at a movie ticket counter. The first element added to the queue will be the first one to be removed.

Tree: A tree consists of a set of elements (nodes) which can be linked to other elements, sometimes hierarchically and sometimes not. A tree data structure can be defined recursively (locally) as a collection of nodes (starting at a root node), where each node is a data structure consisting of a value, together with a list of references to nodes (the 'children'), with the constraint that no reference is duplicated or points to the root.

Earlier, all these data structures were written in C or C++ and had to be created every time in order to store data. Java came up with a solution that provided an API in the *java.util* package. This contains classes and interfaces which implement all these data structures, are ready to be used and make work easy for the coder.

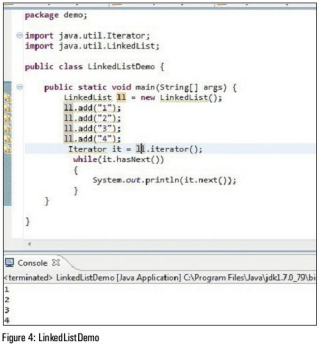
In looking further to find how Java implements the



```
package demo;

import java.util.ArrayList;
import java.util.Iterator;

public class ArrayListDemo {

    public static void main(String args[]) {
        ArrayList al = new ArrayList();
        al.add("Yahoo");
        al.add("Google");
        al.add("Microsoft");
        al.add("Facebook");
        Iterator it = al.iterator();
        while(it.hasNext())
        {
            System.out.println(it.next());
        }

    }
}
```

```
Console
<terminated> ArrayListDemo [Java Application] C:\Program Files\Java\jdk1.7.0_79\bin\javaw
Yahoo
Google
Microsoft
Facebook
```

Figure 3: ArrayListDemo



```
package demo;

import java.util.Iterator;
import java.util.LinkedList;

public class LinkedListDemo {

    public static void main(String[] args) {
        LinkedList ll = new LinkedList();
        ll.add("1");
        ll.add("2");
        ll.add("3");
        ll.add("4");
        Iterator it = ll.iterator();
        while(it.hasNext())
        {
            System.out.println(it.next());
        }
    }

}
```

```
Console
<terminated> LinkedListDemo [Java Application] C:\Program Files\Java\jdk1.7.0_79\bin\j
1
2
3
4
```

Figure 4: LinkedListDemo

structuring of data, I came across the Java Collection framework. This provides the architecture to store and manipulate a group of objects. Operations like searching, sorting, insertion, deletion and manipulation are all performed using the collection framework.

Now let's carefully analyse each data structure and look at how these are implemented by Collection. The Java Collection framework provides many interfaces (Set, List, Queue, Deque, etc) and classes (ArrayList, Vector, LinkedList, HashSet, LinkedHashSet, TreeSet, etc).

ArrayList class: This class uses dynamic arrays for